

R² Gravity v2: Lock-First ModelSpec + Golden Tests

Purpose

Build a **self-validating**, convention-robust toolkit for flat-FLRW $f(R)=R+\alpha R^2$ cosmology.

The key idea: **centralize all conventions** (action normalization, unit mode, scalaron mass scale, frame maps) into a single authoritative object (`ModelSpecR2`), and enforce **golden identity tests** at construction time.

This prevents silent “factor drift” (e.g., 12 vs 6, 4 vs 2) from propagating into observables like A_s, n_s, r .

Design Principles

Lock-first architecture

- **One authoritative definition** of the model (preset + unit mode).
- All derived quantities ($f_R, f_{RR}, m, V_E, R(\phi), \phi(R)$) are computed **only** from that definition.
- Simulation code (e.g., `Background`) **never** hardcodes factors like `12`, `4`, `16`.

Golden tests (fast, brutal, effective)

Run automatically on instantiation via `validate()`:

1. **Inverse-map test:** $R \rightarrow \phi \rightarrow R$ round-trip consistency.
2. **Potential curvature test:** $V_E''(0) = m^2$.
3. **Plateau identity:** $V_0 = \frac{3}{4} M_{\text{pl}}^2 m^2 = \frac{M_{\text{pl}}^2}{8\alpha_{\text{phys}}}$.

Presets and unit modes

Keep presets finite to avoid combinatorics:

- `ALPHA_PHYSICAL`: $f(R) = R + \alpha R^2$, α has dimensions $[\text{mass}]^{-2}$.
- `ALPHA_PLANCK`: $\alpha_{\text{code}} = \alpha_{\text{phys}} M_{\text{pl}}^2$ (dimensionless in reduced-Planck units).
- `STAROBINSKY_M`: $f(R) = R + \frac{R^2}{6M^2}$ (common inflation convention).

Minimal-but-honest extensibility

- Add an **R²-only** FLRW correction helper: Q_{00} for flat FLRW.
 - Optional `custom_fr` mode for general $f(R)$: require user-provided `f`, `f_R`, `f_RR`; allow optional analytic inverse `R_from_phi`; otherwise numeric inversion with warnings.
 - Require stability sampling for custom models: check `f_R>0`, `f_RR>0` on user-declared intervals.
-

Core identities to enforce

Model

$$f(R) = R + \alpha R^2.$$

Scalar-tensor mapping (Jordan frame)

$$\phi \equiv f_R = 1 + 2\alpha R, \quad R = \frac{\phi - 1}{2\alpha}.$$

Scaloron mass (around Minkowski / small curvature)

$$m^2 = \frac{1}{6\alpha_{\text{phys}}}.$$

Einstein-frame potential (Starobinsky form)

$$V_E(\varphi) = \frac{3}{4} M_{\text{pl}}^2 \quad m^2 \left(1 - e^{-\sqrt{\frac{2}{3}} \varphi / M_{\text{pl}}} \right)^2.$$

Plateau height

$$V_0 = \frac{3}{4} M_{\text{pl}}^2 \quad m^2 = \frac{M_{\text{pl}}^2}{8\alpha_{\text{phys}}}.$$

Flat-FLRW R^2 correction term (drop-in helper)

For flat FLRW and pure R^2 correction:

$$Q_0 = 18\alpha_{\text{phys}} (6H^2 \dot{H} + 2H \ddot{H}).$$

Code: `ModelSpecR2` (NumPy-only)

```
from __future__ import annotations
from dataclasses import dataclass
from enum import Enum
import numpy as np

class Preset(Enum):
    # f(R) = R + alpha R^2
    ALPHA_PHYSICAL = "alpha_physical" # alpha has dimensions [mass]^-2
    ALPHA_PLANCK = "alpha_planck"
    # alpha_planck := alpha_physical * Mpl^2 (dimensionless)

    # f(R) = R + R^2/(6 M^2) (common inflation convention)
```

```

STAROBINSKY_M = "starobinsky_M"    # specify M (mass scale)

@dataclass(frozen=True)
class ModelSpecR2:
    preset: Preset
    Mpl: float = 1.0

    # user inputs (only some are used depending on preset)
    alpha: float | None = None    # meaning depends on preset
    M: float | None = None        # only for STAROBINSKY_M

    # validation behavior
    auto_validate: bool = True
    tol_rel: float = 1e-10
    tol_abs: float = 1e-12

    # optional citations + notes
    literature_cite: dict | None = None
    convention_note: str = (
        "R^2 is treated as an EFT correction; results are classical background + minimal
        EF perturbations."
    )

    def __post_init__(self):
        if self.Mpl <= 0:
            raise ValueError("Mpl must be positive.")
        if self.preset == Preset.STAROBINSKY_M:
            if self.M is None or self.M <= 0:
                raise ValueError("Preset STAROBINSKY_M requires M>0.")
        else:
            if self.alpha is None or self.alpha <= 0:
                raise ValueError("Preset ALPHA_* requires alpha>0.")

        if self.auto_validate:
            self.validate()

    # ----- canonical resolved parameters -----
    @property
    def alpha_phys(self) -> float:
        """alpha with dimensions [mass]^2 in c=h=1 units."""
        if self.preset == Preset.ALPHA_PHYSICAL:
            return float(self.alpha)
        if self.preset == Preset.ALPHA_PLANCK:
            # alpha_planck = alpha_phys * Mpl^2 => alpha_phys = alpha_planck / Mpl^2
            return float(self.alpha) / (self.Mpl * self.Mpl)

```

```

    if self.preset == Preset.STAROBINSKY_M:
        #  $f(R)=R + R^2/(6M^2) \Rightarrow \alpha_{\text{phys}} = 1/(6M^2)$ 
        return 1.0 / (6.0 * float(self.M) * float(self.M))
    raise RuntimeError("Unknown preset.")

@property
def m2(self) -> float:
    """Scalaron mass^2 around the minimum."""
    # For  $f(R)=R+\alpha R^2$ :  $m^2 = 1/(6 \alpha_{\text{phys}})$ 
    return 1.0 / (6.0 * self.alpha_phys)

@property
def m(self) -> float:
    return float(np.sqrt(self.m2))

@property
def V_plateau(self) -> float:
    """Plateau height of the EF potential."""
    #  $V_0 = (3/4) M_{\text{pl}}^2 m^2$ 
    return 0.75 * (self.Mpl * self.Mpl) * self.m2

# ----- f(R) and derivatives -----
def f(self, R: float | np.ndarray) -> float | np.ndarray:
    return R + self.alpha_phys * (R * R)

def f_R(self, R: float | np.ndarray) -> float | np.ndarray:
    return 1.0 + 2.0 * self.alpha_phys * R

def f_RR(self, R: float | np.ndarray) -> float | np.ndarray:
    # constant for pure  $R^2$ 
    return 2.0 * self.alpha_phys + 0.0 * np.asarray(R)

# ----- Jordan <-> EF maps -----
def phi_from_R(self, R: float | np.ndarray) -> float | np.ndarray:
    """ $\phi := f_R(R)$ ."""
    return self.f_R(R)

def R_from_phi(self, phi: float | np.ndarray) -> float | np.ndarray:
    """Inverse of phi_from_R for  $R+\alpha R^2$ ."""
    phi = np.asarray(phi)
    return (phi - 1.0) / (2.0 * self.alpha_phys)

def varphi_from_phi(self, phi: float | np.ndarray) -> float | np.ndarray:
    """Canonical EF scalar varphi from  $\phi=f_R$ ."""
    phi = np.asarray(phi)
    if np.any(phi <= 0):
        raise ValueError("phi must be >0 for EF map (log).")
    return np.sqrt(3.0 / 2.0) * self.Mpl * np.log(phi)

```

```

def phi_from_varphi(self, varphi: float | np.ndarray) -> float | np.ndarray:
    varphi = np.asarray(varphi)
    return np.exp(np.sqrt(2.0 / 3.0) * varphi / self.Mpl)

# ----- EF potential -----
def V_E(self, varphi: float | np.ndarray) -> float | np.ndarray:
    """Starobinsky EF potential with resolved m."""
    varphi = np.asarray(varphi)
    y = np.exp(-np.sqrt(2.0 / 3.0) * varphi / self.Mpl)
    return 0.75 * (self.Mpl * self.Mpl) * self.m2 * (1.0 - y) ** 2

# ----- R^2-only flat-FLRW correction helper -----
def Q00_flrw(self, H: float, Hdot: float, Hddot: float) -> float:
    """Flat-FLRW Q_00 for pure R^2 correction:  $Q_{00} = 18 \alpha (6 H^2 \dot{H} - \dot{H}^2 + 2 H \ddot{H})$ ."""
    a = self.alpha_phys
    return 18.0 * a * (6.0 * H * H * Hdot - Hdot * Hdot + 2.0 * H * Hddot)

# ----- validation hooks -----
def validate(self) -> None:
    tol_rel, tol_abs = self.tol_rel, self.tol_abs

    # 1) inverse-map check:  $R \rightarrow \phi \rightarrow R$ 
    Rs = np.array([-0.1, -0.01, 0.0, 0.01, 0.1], dtype=float) /
max(self.alpha_phys, 1.0)
    phis = self.phi_from_R(Rs)
    R_back = self.R_from_phi(phis)
    inv_err = np.max(np.abs(R_back - Rs) / (tol_abs + np.abs(Rs)))
    if not (inv_err <= tol_rel):
        raise AssertionError(f"Inverse-map failed: max relative err = {inv_err:g}")

    # 2) potential curvature at minimum:  $V''(0) = m^2$ 
    # adaptive-ish step size
    h = 1e-6 * max(abs(self.Mpl), 1.0)
    Vp = float(self.V_E(h))
    V0 = float(self.V_E(0.0))
    Vm = float(self.V_E(-h))
    Vpp0 = (Vp - 2.0 * V0 + Vm) / (h * h)
    curv_err = abs(Vpp0 - self.m2) / (tol_abs + abs(self.m2))
    if not (curv_err <= 1e-8):
        raise AssertionError(
            f" $V''(0)$  mismatch: Vpp0={Vpp0:g}, m2={self.m2:g},
relerr={curv_err:g}"
        )

    # 3) plateau consistency:  $V_0 = (3/4) M_{\text{pl}}^2 m^2 = M_{\text{pl}}^2 / (8 \alpha_{\text{phys}})$ 

```

```

        V0_expected = (self.Mpl * self.Mpl) / (8.0 * self.alpha_phys)
        plat_err = abs(self.V_plateau - V0_expected) / (tol_abs +
abs(V0_expected))
        if not (plat_err <= tol_rel):
            raise AssertionError(
                f"Plateau mismatch: got {self.V_plateau:g}, exp
{V0_expected:g}, relerr={plat_err:g}"
            )

    def describe(self) -> str:
        lines = [
            f"ModelSpecR2(preset={self.preset}, Mpl={self.Mpl:g})",
            f"Resolved: alpha_phys={self.alpha_phys:g} [mass^-2]",
            f"Resolved: m^2={self.m2:g}, m={self.m:g}",
            f"Resolved: V_plateau=(3/4) Mpl^2 m^2 = {self.V_plateau:g}",
            f"Note: {self.convention_note}",
        ]
        if self.literature_cite:
            lines.append(f"Cites: {self.literature_cite}")
        return "\n".join(lines)

```

Integration into Background (patch plan)

Goal

Make `Background` dynamics **mode-agnostic** and **convention-free** by routing all convention-sensitive quantities through `model_spec`.

Replace hardcoded factors with model calls

- Replace `self.alpha`, `m_from_alpha`, `R_from_phi`, manual R-mode IC maps with:
- `model.alpha_phys`, `model.m`, `model.R_from_phi(phi)`, `model.phi_from_R(R0)`.
- Replace any internal `Q00_R2(...)` helpers with `model.Q00_flrw(H,Hdot,Hddot)`.

Minimal initialization example

```

model = ModelSpecR2(
    preset=Preset.ALPHA_PLANCK,
    alpha=1e9,
    Mpl=1.0,
    literature_cite={"preset": "Starobinsky R^2 (1980)", "note":
"alpha_planck = alpha_phys*Mpl^2"},
)
print(model.describe())

```

```
bg = Background(model_spec=model, mode="phi", **kwargs)
```

Optional: Custom f(R) mode (opt-in)

Contract

Allow user-defined models via a separate class (recommended) or a guarded `custom_fr=True` option.

Minimum required:

- `f(R)`, `f_R(R)`, `f_RR(R)`

Optional:

- analytic inverse `R_from_phi(phi)`

If `R_from_phi` is missing:

- numeric inversion is allowed **only** on a user-declared interval where `f_R` is monotone;
- fallback methods: bracketed bisection if Newton fails.

Stability sampling (mandatory for custom):

- check `f_R(R)>0` and `f_RR(R)>0` on a grid of test points.

Validation Checklist (fast path)

1. ****Instantiate **ModelSpecR2(auto_validate=True)** for each preset and confirm `validate()` passes.
2. Confirm equivalence across presets (same `alpha_phys` $\Rightarrow$ same `m`, `V_plateau`).
3. Patch `Background` to consume `model_spec` and remove hardcoded factors.
4. Re-run EF diagnostics (n_s, r, A_s) and verify Starobinsky relations.
5. Compute `Q00_flrw` and compare to analytic FLRW expression for sanity cases (de Sitter, radiation).

Expected outcomes

With $M_{\text{pl}} = 1$ and $\alpha_{\text{phys}} = 10^9$:

- $m^2 = 1/(6\alpha) = 1.666 \dots \times 10^{-10}$
- $V_0 = (3/4)m^2 = 1.25 \times 10^{-10}$

These should match across ALPHA_PLANCK and STAROBINSKY_M when mapped consistently.

External references

Foundational R^2 / Starobinsky inflation

- A. A. Starobinsky, *A new type of isotropic cosmological models without singularity*, **Phys. Lett. B** **91** (1980) 99–102. DOI: 10.1016/0370-2693(80)90670-X.
- (Context / modern perspective) A. D. Linde, *Alexei Starobinsky and Modern Cosmology*, arXiv: 2509.01675.

$f(R)$ gravity reviews and scalar–tensor equivalence

- A. De Felice, S. Tsujikawa, *$f(R)$ theories*, **Living Rev. Rel.** **13** (2010) 3. DOI: 10.12942/lrr-2010-3. arXiv: 1002.4928.
- T. P. Sotiriou, V. Faraoni, *$f(R)$ theories of gravity*, **Rev. Mod. Phys.** **82** (2010) 451–497. DOI: 10.1103/RevModPhys.82.451. arXiv:0805.1726.

Gauge-invariant cosmological perturbations (Mukhanov–Sasaki)

- V. F. Mukhanov, *Gravitational instability of the universe filled with a scalar field*, **JETP Lett.** **41** (1985) 493–496.
- M. Sasaki, *Large scale quantum fluctuations in the inflationary universe*, **Prog. Theor. Phys.** **76** (1986) 1036–1046.
- V. F. Mukhanov, H. A. Feldman, R. H. Brandenberger, *Theory of cosmological perturbations*, **Phys. Rept.** **215** (1992) 203–333. DOI: 10.1016/0370-1573(92)90044-Z.

Observational constraints used for benchmarks

- Planck Collaboration, *Planck 2018 results. X. Constraints on inflation*, arXiv:1807.06211.
- BICEP/Keck Collaboration, *BICEP2 / Keck Array X: Constraints on primordial gravitational waves using Planck, WMAP, and new BICEP2/Keck observations through the 2015 season*, arXiv:1810.05216.

EFT framing (useful when extending the perturbations layer)

- C. Cheung, P. Creminelli, A. L. Fitzpatrick, J. Kaplan, L. Senatore, *The Effective Field Theory of Inflation*, **JHEP** **03** (2008) 014. arXiv:0709.0293.
- S. Weinberg, *Effective field theory for inflation*, **Phys. Rev. D** **77** (2008) 123541. DOI: 10.1103/PhysRevD.77.123541. arXiv:0804.4291.

Weak-field / Yukawa-limit context (for the “1/3 factor” sanity checks)

- T. Chiba, *1/R gravity and scalar-tensor gravity*, **Phys. Lett. B** **575** (2003) 1–3. arXiv:astro-ph/0307338.
- C. P. L. Berry, J. R. Gair, *Linearized $f(R)$ Gravity: Gravitational Radiation & Solar System Tests*, arXiv: 1104.0819.

Software context (when cross-validating spectra)

- J. Lesgourgues, *The Cosmic Linear Anisotropy Solving System (CLASS) I: Overview*, arXiv:1104.2932.
- D. Blas, J. Lesgourgues, T. Tram, *CLASS II: Approximation schemes*, **JCAP** **07** (2011) 034. DOI: 10.1088/1475-7516/2011/07/034. arXiv:1104.2933.
- A. Lewis, A. Challinor, A. Lasenby, *Efficient Computation of CMB anisotropies in closed FRW models* (CAMB lineage), **Astrophys. J.** **538** (2000) 473–476. DOI: 10.1086/309179. arXiv:astro-ph/9911177.

Optional: recent/adjacent (not required for v2, but relevant)

- I. Asiain, A. Dobado, D. Espriu, *Unitarization of $R + \alpha R^2$ gravity*, arXiv:2512.05911.
- S. Toyama, *Starobinsky inflation beyond the leading order*, **Phys. Rev. D** **110** (2024) 063552. DOI: 10.1103/PhysRevD.110.063552.

Citizen Gardens © 2025 CC-NC-ND 4.0